

Understanding Algorithm Efficiency and Scalability: Randomized Quicksort and Hashing With Chaining

Sanya Kamra

University of the Cumberland

MSCS-532: Algorithms and Data Structures

September 27, 2026

Understanding Algorithm Efficiency and Scalability: Randomized Quicksort and Hashing With Chaining

Randomized Quicksort: Implementation

quicksort.py implements randomized and deterministic Quicksort. The randomized version chooses an index uniformly from the **current** subarray using `randint(lo, hi)`. The deterministic version takes the first element of the subarray. Both versions use Dutch-national-flag (three-way) partitioning: elements less than the pivot, equal to the pivot, and greater than the pivot. The implementations return new lists, preserve their inputs, handle empty/singleton lists, duplicates and negative values, and use an explicit stack rather than recursive Python calls. Each partition scans its subarray in linear time. The explicit stack, with the smaller partition processed immediately, uses $O(\log n)$ auxiliary stack space; the copied input uses $O(n)$ additional space.

Why Randomized Quicksort Has Expected $O(n \log n)$ Time

Assume initially that the n input keys are distinct. Label the sorted keys $z_1 < z_2 < \dots < z_n$. Define X_{ij} as 1 if z_i and z_j are compared directly by the algorithm, and 0 otherwise. The two keys are compared exactly when the first pivot selected from the rank interval $[i, j]$ is one of those two keys. Because pivots are uniformly selected, $P(X_{ij} = 1) = 2/(j-i+1)$. By linearity of expectation, the expected number of comparisons is: (Cormen et al., 2022).

$$\begin{aligned} E[C_n] &= \sum_{1 \leq i < j \leq n} E[X_{ij}] \\ &= \sum_{i < j} 2/(j-i+1) \\ &= 2 \sum_{d=1}^{n-1} (n-d)/(d+1) \\ &= 2(n+1)H_n - 4n, \end{aligned}$$

where $H_n = \sum_{k=1}^n 1/k$ is the n th harmonic number. Since $H_n = \Theta(\log n)$, $E[C_n] = \Theta(n \log n)$. Each partition performs linear work in the subarray length, so this also establishes expected $\Theta(n \log n)$ runtime under the standard unit-cost comparison model.

Equivalently, one can write $E[T(n)] = (1/n) \sum_{k=0}^{n-1} (E[T(k)] + E[T(n-k-1)]) + \Theta(n)$, which solves to $\Theta(n \log n)$.

Important implementation nuance: The exact comparison-count formula above is for the conventional distinct-key, two-way comparison model. Our three-way implementation makes additional branch comparisons per inspected element, but has the same expected $\Theta(n \log n)$ complexity for distinct keys. Three-way partitioning can do better when many keys are equal. Randomization does **not** eliminate the quadratic worst case: consistently extreme pivots still yield $O(n^2)$ runtime, although that pivot pattern becomes unlikely on large distinct-key arrays.

Empirical Quicksort Comparison

Method

benchmark.py generates distinct random arrays sampled without replacement, increasing arrays, decreasing arrays, and arrays with ten possible values. Sizes are 100, 500, 1,000, 2,000, and 4,000. Five timed trials run for each size/distribution/algorithm combination. The same input is used for both algorithms within each trial, and outputs are asserted equal to Python's sorted. The script records mean, median, minimum and maximum wall-clock duration in milliseconds using time.perf_counter_ns. Input generation and correctness verification are **outside** the measured interval. Random number generators use fixed seeds, although execution times still depend on CPU and system load.

Table 1

Mean Quicksort Runtime by Distribution and Input Size (Milliseconds)

Distribution	n	Randomized (ms)	First-element deterministic (ms)
Random	100	0.0566	0.0499

Random	500	0.3044	0.2413
Random	1,000	0.6540	0.5554
Random	2,000	1.2911	1.0761
Random	4,000	2.7388	2.1832
Sorted	100	0.0466	0.0516
Sorted	500	0.2442	0.4169
Sorted	1,000	0.5344	1.1221
Sorted	2,000	1.1115	2.8822
Sorted	4,000	2.4407	7.9480
Reverse	100	0.0492	0.1562
Reverse	500	0.2454	3.2129
Reverse	1,000	0.5220	15.6287
Reverse	2,000	1.1705	53.7234
Reverse	4,000	2.4833	223.2245
Repeated	100	0.0249	0.0258
Repeated	500	0.0641	0.0693
Repeated	1,000	0.1220	0.1262
Repeated	2,000	0.2227	0.2309
Repeated	4,000	0.4133	0.4400

The values in Table 1 are the mean runtimes printed by the benchmark on Sanya Kamra's MacBook on September 27, 2026. Each displayed number aggregates five timed trials. The script writes the corresponding detailed trial summaries, including medians and ranges, to `results/timings.csv` when run locally. The previous bundled CSV reflects a different machine and should not be presented as these local measurements unless replaced by the locally generated CSV.

Interpretation

For randomly ordered **distinct** inputs, both versions generally exhibit $n \log n$ scaling. The deterministic version is somewhat faster in these local measurements because selecting a random pivot adds random-number-generator overhead and the random inputs do not systematically disadvantage the first-pivot choice. This does not mean deterministic first-pivot Quicksort has a better worst-case bound.

For sorted and reverse-sorted arrays, deterministic first-pivot choice can create highly unbalanced partitions. Reverse-sorted arrays show the clearest quadratic-like growth in the local MacBook benchmark: at $n = 4,000$, deterministic execution takes about 223.22 ms versus 2.48 ms for randomized execution (approximately 90 times as long in this run). **The deterministic sorted-array timings are less extreme than a textbook naive first-pivot Quicksort:** in-place three-way partitioning rearranges values during scanning, so it can disrupt the initial order of recursive subproblems. It remains sensitive to input ordering, but its exact timings depend on partition implementation.

Repeated-element arrays are fast with *both* variants because three-way partitioning removes the entire equal-key group from further partitioning. The standard distinct-key analysis therefore does not predict their exact runtime. Small samples, interpreter overhead, timing noise,

and the randomized pivot's extra work can reverse small timing differences; reproducibility is improved by five trials and reporting both mean and median, but stronger claims would require larger trials and additional machines.

Hashing with Chaining: Implementation

hash_table.py implements a hash table for **integer keys**, including negative keys, with arbitrary Python values. Each bucket is a Python list of (key, value) pairs. The initialized hash function is selected at random from the family

$$h_{(a,b)}(k) = ((a \cdot k + b) \bmod p) \bmod m,$$

where $p = 2^{61} - 1$, a is uniform in $\{1, \dots, p-1\}$, b is uniform in $\{0, \dots, p-1\}$, and m is the current number of buckets. This is based on a universal hashing construction over integer keys considered modulo p ; keys congruent modulo p cannot be distinguished by this hash family. The reduction to arbitrary m can also introduce small distribution differences. A prime modulus larger than the expected bounded key universe is preferable when strict theoretical universality assumptions are required.

- `insert(key, value)`: updates an existing key or appends a new pair.
- `search(key)`: returns the associated value or raises `KeyError` when absent, distinguishing absence from a stored `None`.
- `delete(key)`: removes a key or raises `KeyError` when absent.

The table doubles the bucket count **before** a new insertion would exceed a 0.75 load factor. It halves the bucket count when the load factor falls below 0.20, subject to the initial minimum capacity. On resize, stored entries are rehashed because m changes; all original key-value associations remain intact.

Expected Complexity and Load Factor

Define the load factor $\alpha = n/m$, where n is the number of stored entries and m is the number of bucket slots. Under **simple uniform hashing**, each key is equally likely to be hashed to any of the m slots independently of the other keys. Expected chain length is α . (Cormen et al., 2022).

Table 2

Expected Hash-Table Operation Costs Under Simple Uniform Hashing

Operation	Expected runtime without resizing	Explanation
Search, unsuccessful	$\Theta(1 + \alpha)$	Hash the key and scan its chain, whose expected length is α .
Search, successful	$\Theta(1 + \alpha)$	Hash and scan until the matching pair is encountered.
Insert, new key	$\Theta(1 + \alpha)$	Search for an existing key, then append to the chain.
Insert, existing key	$\Theta(1 + \alpha)$	Find and update the matching pair.
Delete	$\Theta(1 + \alpha)$	Search for the matching pair and remove it from its chain.

A resize costs $\Theta(n + m_{\text{new}})$ time to allocate buckets and rehash all entries. This is an **occasional worst-case cost**, not a constant-time guarantee per operation. Geometric growth

(doubling) distributes that cost across many insertions, giving expected **amortized** $O(1)$ insertion when α is bounded and ordinary hashing assumptions hold. A small load factor reduces average chain length at the cost of more unused memory; a large load factor saves memory but increases expected search and deletion time. Shrink and grow thresholds are separated to avoid repeatedly resizing near a single threshold (resize thrashing).

The theoretical $O(1 + \alpha)$ claim relies on the hashing assumptions. It does not provide a worst-case guarantee: if all keys fall into one chain, lookup/deletion can take $O(n)$. Randomly selecting a hash function mitigates predictable collisions for a suitable key universe, but does not prevent maliciously chosen or congruent-mod- p keys from colliding.

Test Evidence and Reproducibility

The local terminal output shows three successful unit tests: hash collision/validation, hash insert-search-update-delete, and Quicksort edge cases. The unittest runner reports “Ran 3 tests in 0.002s” and “OK.” The separate benchmark terminal output shows all four distributions across five sizes and confirms that results/timings.csv was saved to the local project directory. The screenshots below document the execution rather than replacing the analysis.

Conclusion

Random pivot selection improves robustness against input-order patterns for distinct keys, while adding modest runtime overhead on already random arrays. Three-way partitioning is particularly beneficial for repeated keys. Separate chaining supports expected constant-time dictionary operations when the load factor remains bounded, at the cost of $O(n)$ individual resize events and extra bucket memory. Together, these experiments show why algorithm choice requires both asymptotic analysis and implementation-aware measurements.

Code Repository

The complete Python implementation, tests, benchmark script, results, and README are available in the project repository: https://github.com/sanyakamra96/MSCS532_Assignment3

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Kamra, S. (2026). MSCS532_Assignment3 [Computer software]. GitHub.
https://github.com/sanyakamra96/MSCS532_Assignment3

Python Software Foundation. (n.d.). *time* — *Time access and conversions*.
<https://docs.python.org/3/library/time.html>

Appendix A. Local Execution Screenshots

Figure A1

Unit-test execution on the local MacBook: all three tests pass.

```
sanyakamra@Sanyas-MacBook-Air MSCS532_Assignment3 % python3 -m unittest -v
test_hash_collision_and_validation (test_algorithms.AlgorithmTests.test_hash_collision_and_validation) ... ok
test_hash_insert_search_update_delete (test_algorithms.AlgorithmTests.test_hash_insert_search_update_delete) ..
ok
test_sort_edge_cases (test_algorithms.AlgorithmTests.test_sort_edge_cases) ... ok

-----
Ran 3 tests in 0.002s

ok
```

Figure A2

Local benchmark output: mean Quicksort times (ms), four input distributions, five input sizes, and CSV output confirmation.

```
sanyakamra@Sanyas-MacBook-Air MSCS532_Assignment3 % python3 benchmark.py
random    n= 100 randomized= 0.0566 ms deterministic= 0.0499 ms
random    n= 500 randomized= 0.3044 ms deterministic= 0.2413 ms
random    n= 1000 randomized= 0.6540 ms deterministic= 0.5554 ms
random    n= 2000 randomized= 1.2911 ms deterministic= 1.0761 ms
random    n= 4000 randomized= 2.7388 ms deterministic= 2.1832 ms
sorted    n= 100 randomized= 0.0466 ms deterministic= 0.0516 ms
sorted    n= 500 randomized= 0.2442 ms deterministic= 0.4169 ms
sorted    n= 1000 randomized= 0.5344 ms deterministic= 1.1221 ms
sorted    n= 2000 randomized= 1.1115 ms deterministic= 2.8822 ms
sorted    n= 4000 randomized= 2.4407 ms deterministic= 7.9480 ms
reverse   n= 100 randomized= 0.0492 ms deterministic= 0.1562 ms
reverse   n= 500 randomized= 0.2454 ms deterministic= 3.2129 ms
reverse   n= 1000 randomized= 0.5220 ms deterministic= 15.6287 ms
reverse   n= 2000 randomized= 1.1705 ms deterministic= 53.7234 ms
reverse   n= 4000 randomized= 2.4833 ms deterministic= 223.2245 ms
repeated  n= 100 randomized= 0.0249 ms deterministic= 0.0258 ms
repeated  n= 500 randomized= 0.0641 ms deterministic= 0.0693 ms
repeated  n= 1000 randomized= 0.1220 ms deterministic= 0.1262 ms
repeated  n= 2000 randomized= 0.2227 ms deterministic= 0.2309 ms
repeated  n= 4000 randomized= 0.4133 ms deterministic= 0.4400 ms
Saved /Users/sanyakamra/Downloads/MSCS532_Assignment3/results/timings.csv
```